

Звіти з лабораторних робіт з дисципліни «Сучасна криптографія»

Виконав: студент(ка) проєкту MCL2

2026

Зміст

1	Лабораторна робота №1: Симетричні шифри та криптоаналіз повторюваного XOR	2
1.1	Мета роботи	2
1.2	Теоретичні відомості	2
1.3	Опис реалізації	2
1.4	Висновок	2
2	Лабораторна робота №2: Блочне шифрування. Стандарт AES та режими його роботи	3
2.1	Мета роботи	3
2.2	Теоретичні відомості	3
2.3	Опис реалізації	3
2.4	Висновок	3
3	Лабораторна робота №3: Криптографічні хеш-функції. SHA-256, лавинний ефект та пошук колізій	4
3.1	Мета роботи	4
3.2	Теоретичні відомості	4
3.3	Опис реалізації	4
3.4	Висновок	4
4	Лабораторна робота №4: Асиметрична криптографія. Реалізація RSA та схеми оптимального асиметричного шифрування із доповненням (OAEP)	5
4.1	Мета роботи	5
4.2	Теоретичні відомості	5
4.3	Опис реалізації	5
4.4	Висновок	5

1 Лабораторна робота №1: Симетричні шифри та криптоаналіз повторюваного XOR

Файли проєкту: `task1.py`, `task2.py`, `task3.py`, `utils.py`, `L1.py`.

1.1 Мета роботи

Ознайомлення з базовими принципами симетричного шифрування, вивчення властивостей операції XOR (виключного АБО) та реалізація методів криптоаналізу для шифрів із повторюваним ключем (симетричний криптоаналіз).

1.2 Теоретичні відомості

Операція XOR ($\oplus$) є основою багатьох криптосистем. Проте використання короткого повторюваного ключа (шифр Віженера / Repeating-key XOR) є вразливим до статистичного аналізу. Використовуючи індекс збігів (Index of Coincidence) або відстань Хеммінга, можна визначити довжину ключа L . Після цього шифротекст розбивається на L блоків, кожен з яких аналізується незалежно як шифр з однобайтовим XOR за допомогою частотного аналізу символів цільової мови.

1.3 Опис реалізації

- `task1.py`: Реалізація базового однобайтового XOR-шифрування та його автоматичного зламу методом підбору ключа (*brute-force*) за допомогою статистичної оцінки частоти літер.
- `task2.py`: Реалізація шифрування довільного тексту методом повторюваного багатовбайтового XOR (*Multi-byte / Repeating-key XOR*).
- `task3.py`: Криптоаналіз повторюваного XOR без апіорного знання ключа. Визначення довжини ключа за допомогою відстані Хеммінга та наступне відновлення початкового тексту.
- `utils.py` та `L1.py`: Допоміжні функції для якісної конвертації форматів даних (Hex, Base64, Bytes) та головна точка входу для демонстрації результатів роботи модулів.

1.4 Висновок

У ході роботи було досліджено слабкості повторюваного XOR. Доведено, що симетричні шифри з фіксованою довжиною ключа, який циклічно повторюється, є абсолютно ненадійними проти частотного криптоаналізу, навіть якщо сам простір відкритих текстів є великим.

2 Лабораторна робота №2: Блочне шифрування. Стандарт AES та режими його роботи

Файли проєкту: `aes.py`, `constants.py`, `analysis.py`, `utils.py`, `L2.py`.

2.1 Мета роботи

Дослідження архітектури сучасного стандарту блочного шифрування AES (*Advanced Encryption Standard*), вивчення режимів роботи блочних шифрів та аналіз їхньої криптографічної стійкості.

2.2 Теоретичні відомості

AES — це симетричний блочний шифр із розміром блока 128 біт. Він базується на структурі підстановочно-перестановочної мережі (SPN) і складається з кількох раундів (10, 12 або 14 залежно від ключа). Кожен раунд містить такі трансформації:

1. **SubBytes** — нелінійна заміна кожного байта через фіксовану таблицю S-Box.
2. **ShiftRows** — циклічний зсув рядків стану матриці на різні зміщення.
3. **MixColumns** — дифузія всередині стовпців шляхом множення над скінченним полем Галуа $GF(2^8)$.
4. **AddRoundKey** — накладання поточного раундового ключа за допомогою операції XOR.

Для шифрування повідомлень, більших за розмір одного блока, використовуються режими: ECB (найочевидніший, але незахищений), CBC (із вектором ініціалізації IV та зачепленням блоків), CTR (потоківий режим) тощо.

2.3 Опис реалізації

- `aes.py`: Програмна реалізація ядра алгоритму AES (процедура розширення ключа *KeyExpansion*, прямі та зворотні раундові трансформації).
- `constants.py`: Статичні таблиці заміни S-Box, Inverse S-Box, а також константи раундів (*Rcon*).
- `analysis.py`: Модуль для дослідження властивостей шифру. Містить аналіз витoku інформації при використанні режиму ECB (збереження контурів на структурах даних) або перевірку лавинного ефекту.
- `L2.py`: Запуск тестів шифрування та дешифрування для різних режимів (зокрема CBC) та верифікація коректності відповідно до офіційних тестів NIST (*Known Answer Tests*).

2.4 Висновок

Було детально вивчено внутрішню математичну структуру AES. Практично підтверджено, що правильний вибір режиму роботи блочного шифру (наприклад, CBC замість ECB) є критично важливим для забезпечення конфіденційності, оскільки ECB не приховує статистичні закономірності відкритих даних.

3 Лабораторна робота №3: Криптографічні хеш-функції. SHA-256, лавинний ефект та пошук колізій

Файли проєкту: `sha256_core.py`, `constants.py`, `collisions.py`, `avalanche.py`, `L3.py`.

3.1 Мета роботи

Вивчення принципів побудови криптографічних хеш-функцій на прикладі алгоритму SHA-256, дослідження лавинного ефекту та практична демонстрація складності пошуку колізій (атака «днів народження»).

3.2 Теоретичні відомості

Криптографічна хеш-функція повинна задовольняти три основні вимоги: стійкість до пошуку першого образу (незворотність), стійкість до пошуку другого образу та стійкість до колізій (задача знаходження двох довільних повідомлень $x \neq y$ таких, що $H(x) = H(y)$). Алгоритм SHA-256 використовує структуру Меркла-Дамґорда, оперує 32-бітними словами та застосовує побітові логічні операції (*AND*, *XOR*, *ROTR*, *SHR*) протягом 64 раундів стиснення.

3.3 Опис реалізації

- `sha256_core.py`: Низькорівнева реалізація алгоритму SHA-256: доповнення повідомлення (*padding*), ініціалізація векторів стану $H_0 \dots H_7$, циклічні зсуви та функції раундів Ch , Maj , Σ_0 , Σ_1 .
- `avalanche.py`: Скрипт для дослідження лавинного ефекту (*Avalanche effect*). Вимірює відсоток зміни бітів вихідного хешу при зміні лише 1 біта у вхідному рядку (ідеальним є показник $\approx 50\%$).
- `collisions.py`: Реалізація алгоритму пошуку колізій для урізаного варіанта SHA-256 (перші 16–32 біти виходу) з використанням парадоксу днів народження.
- `L3.py`: Головний модуль для координації експериментів та візуалізації отриманих результатів.

3.4 Висновок

Під час лабораторної роботи було реалізовано SHA-256 та експериментально підтверджено наявність сильного лавинного ефекту. Пошук колізій для повнорозмірного хешу є обчислювально неможливим, проте для урізаних версій колізії знаходяться за усереднений час $\mathcal{O}(2^{n/2})$, що наочно демонструє сутність атаки «днів народження».

4 Лабораторна робота №4: Асиметрична криптографія. Реалізація RSA та схеми оптимального асиметричного шифрування із доповненням (OAEP)

Файли проєкту: `primes.py`, `rsa.py`, `oaep.py`, `L4.py`.

4.1 Мета роботи

Дослідження принципів криптографії з відкритим ключем, практична реалізація алгоритму RSA та інтеграція стандарту доповнення шифру PKCS#1 v2.0 (OAEP) для забезпечення стійкості проти активних атак.

4.2 Теоретичні відомості

Стійкість криптосистеми RSA базується на складності задачі факторизації великих цілих чисел. Математична модель описується так:

$$n = p \cdot q \quad (1)$$

Відкрита експонента e та секретна експонента d пов'язані математичним співвідношенням через функцію Ейлера $\phi(n)$:

$$e \cdot d \equiv 1 \pmod{\phi(n)} \quad (2)$$

Базовий варіант RSA (*Textbook RSA*) є детермінованим і вразливим до багатьох типів атак. Для безпечного використання на практиці застосовують схему **OAEP** (*Optimal Asymmetric Encryption Padding*), яка перетворює шифр на ймовірнісний (*probabilistic*) та гарантує стійкість класу IND-CCA2 (стійкість до атак на основі підбраного шифротексту) за допомогою маскувальних функцій (*MGF1*).

4.3 Опис реалізації

- `primes.py`: Алгоритми теорії чисел: генерація великих простих чисел, ймовірнісний тест простоти Міллера-Рабіна, розширений алгоритм Евкліда для обчислення модульного оберненого елемента (d).
- `rsa.py`: Реалізація генерації пари ключів `generate_keypair()`, математичні операції шифрування $c = m^e \pmod{n}$ та дешифрування $m = c^d \pmod{n}$.
- `oaep.py`: Реалізація протоколу доповнення OAEP перед шифруванням. Включає додавання випадкового вектора (*seed*), розширення маски через *MGF1* та каскадні операції XOR.
- `L4.py`: Інтеграція всіх модулів та фінальна демонстрація безпечного процесу передачі даних.

4.4 Висновок

У ході роботи було побудовано повноцінну асиметричну систему шифрування. Додавання шару OAEP дозволило захистити алгоритм RSA від структурних атак на відкритий текст та забезпечити повну відповідність сучасним криптографічним вимогам безпеки.